

Prague University of Economics and Business
Faculty of Informatics and Statistics



Application of multivariate statistical methods for football player analysis

BACHELOR THESIS

Study program: Mathematical Methods in Economics

Specialization: Data Analyses and Modeling

Author: Igor Kalashnikov

Supervisor: Ing. Jan Fojtík, Ph.D

Prague, May 2024

Acknowledgements

I want to thank my supervisor Ing. Jan Fojtík, Ph.D for giving an useful guidance throughout all the time thesis has been written, and Bc.Vladimir Kopylov for advice from own experience that helped me organize my work.

Abstract

Bachelor thesis is aimed to give an overview, how multivariate statistical methods could be used in football player analysis and decision making by team management departments while considering a player replacement. Firstly, Principal Component Analysis is presented as a method for dimension reduction and highlighting most common patterns in player statistics. Then, different similarity/dissimilarity metrics applied to transformed data in order to measure, how much players differ from each other. Cluster analysis is presented and applied for unionizing players into groups of similar ones. For the better presentation of results, there is a basic Shiny app developed and deployed, that has functionality useful as for specialists in analytics in statistics, as for stakeholders in order to get a quick user-friendly overview of possible candidates for player replacement and different trends visualization.

Keywords

sports statistics, data analysis, Shiny app, principal component analysis, football player replacement model, cluster analysis

Contents

Introduction	7
1 Data preprocessing	8
1.1 Web scraping	8
1.2 Package worldfootballR	8
1.3 Data import and preparation	8
1.4 Data dictionary	10
1.5 Values standardization	13
1.6 Position specificity	14
2 Principal Component Analysis	15
2.1 Covariance matrix	15
2.2 Eigenvalues and eigenvectors	16
2.3 Principal components	16
2.4 PCA outputs	17
2.4.1 Variable to component correlation	17
2.4.2 Variables contributions	17
2.4.3 Variance explained	17
3 Measurement of similarity between objects	18
3.1 Similarity metrics	18
3.2 Dissimilarity metrics	18
3.3 Euclidean similarity matrix	19
4 Cluster analysis	20
4.1 Hierarchical clustering	20
4.2 Partitional clustering, k-means algorithm	21
4.3 Optimal number of clusters determination	22
4.3.1 Rule of thumb	22
4.3.2 Elbow method	22
4.3.3 Silhouette method	23
4.3.4 Midfielders dataset optimal number of clusters	24
5 Player replacement model	26
5.1 Tuning model - age and price filters	26
5.2 Central defenders PCA application	27
5.3 K-means clustering for defenders	28
5.4 Similarity matrix - Ming-jae's replacement candidates	30
5.5 Breakdown to different stat types	32
6 Shiny web application	34

6.1	Shiny environment	34
6.2	Main Shiny concepts	35
6.2.1	User interface module	35
6.2.2	Server module	37
6.2.3	App script structure	38
6.3	Web applications publishing	39
	Conclusion	40
	Literature	41

List of Figures

4.1	Example of cluster analysis viz to determine 'elbow' point. Own processing with R	23
4.2	Chart to determine 'elbow point' for optimal number of clusters in midfielders dataset. Own processing with R	25
5.1	Bar chart, showing amounts of players assigned to each cluster. Own processing with R	29
5.2	Scatter plot, revealing distribution of players by clusters. Kim Min-jae is highlighted as a data point with bigger size. Own processing with R	30
5.3	Kim Min-jae vs Edmond Tapsoba stats comparison on a radar plot. Own processing with R	31
6.1	Shiny application structure - different modules with their location.[20]	35
6.2	Sidebar appearance of the Shiny app for player replacement model. Own screenshot from R Shiny app	36
6.3	Main panel appearance of the Shiny app for player replacement model. Own screenshot from R Shiny app	37

List of Tables

1.1	Data dictionary	13
4.1	Comparison of WCSS and average silhouette scores after running k-means algorithm with different number of clusters	24
5.1	Descriptive statistics of transfer fee and age columns in dataset of Napoli arrivals last 5 years	26
5.2	Principal component loadings for three most contributed variables	27
5.3	Sample of 4 records from defenders similarity matrix	30
5.4	Top five replacements for Kim Min-jae by similarity	31
5.5	Top five replacements for Kim Min-jae - comparison between top with filters for age and price applied and without them	32
5.6	Overview of variables with highest correlation to each of principle components - only passing stats analyzed	33
5.7	Top five replacements for Kim Ming-jae - by passing and possession stats	33

Introduction

People's interest in professional sports has intensified significantly in last decades. Despite local fans always had a possibility to come to support their team on the stadium, new media technologies development (radio, TV, internet) allowed people all over the world get updates and follow live streams of any sport event imaginable easier and easier. Watching Champions league football on Tuesdays and Wednesdays, or Formula 1 Grand Prix on Sunday night has become a very popular form of entertainment, even a family tradition for someone. With such increased popularity, sports teams revenues increased as well, and therefore, way how club owners handle their business affairs became very important and influential on the sport side.

Every sport in their basis was always about various figures and statistics. It could be number of goals scored in a football match, or number of laps lead in a race, or even average attendance of an event – for all these stats could be found a useful application. With modern techniques of data capturing, one football match could provide huge amount of data, from the overall team stats of shooting, passing, ball possession to movements of each of more than 20 players participated during the whole game. Based on analyses of player/team performance, important decisions could be made by team on how to improve in future. Concept of 'Moneyball' [1], introduced and successfully launched by baseball team Oakland Athletics, became an inspiration for a lot of clubs in different sports. Idea of using data and statistical research in order to complete own squad with decent sportsmen while having limited budget, and increase that budget by earnings from selling their successfully players, is widely used by English side Brentford F.C. [2] Positive result of this approach is visible, as team, that finished ninth in the third English tier year before club was bought by Bernham, in 2013, in following 10 years made their move to become a pretty strong mid-table team in Premier League, once reached semi final of English Cup in 20/21 season.

One of the possible application of statistical methods on players data is an implementation of algorithm, that could study patterns and similarities between footballers in order to make a list of potential candidates who could be a good replacement for some given player. This thesis would be aimed on developing basic 'player replacement model', and some statistical concepts were chosen:

- Principal component analysis – as original dataset would have a lot of variables, with risk of multicollinearity, so this technique would help to reduce amount of dimensions keeping most of the information from original data.
- Similarity measures – in order to determine similar objects, there should be calculated any similarity/dissimilarity metric, that would show how observations differ from each other.
- Cluster analysis - observations from dataset could also be divided into groups, each of them having specific set of attributes, that is very identical for players represent same group and diverge across clusters.

1. Data preprocessing

As main goal of the thesis is to analyze similarity of different football player stats, and implement an algorithm that will be able to find possible replacements for each individual, there was a specific dataset chosen as a base for the analysis. This chapter covers all the steps done in order to get relevant data, starting from descriptions of technologies used (code in R programming language, especially package `worldfootballR`), actions made to clean and prepare data, and a data dictionary that describes each variable.

1.1 Web scraping

Each study, that requires any practical experiment to be done, requires some good quality data. In case of football stats, there is a lot of information shared in public in the internet, therefore it is a good choice to use web scraping methods to retrieve relevant data for that analysis. Shortly, web scraping is a technique that allows user to extract available data from websites and transforming that data into structures that can be stored and analyzed easily (dataframes in R or Python, SQL/no-SQL databases, etc.). There are multiple specific methods, how to implement a scraping, and one of the most popular is using Hyper Text Markup Language (HTML) structure of the website. [3] Based on inspection of web pages elements it is possible to select relevant ones that should be extracted in purpose of the analysis.

1.2 Package `worldfootballR`

All the data used in this thesis were retrieved using one of the user-made package for programming language R, `worldfootballR` [4]. Functions built inside that package allow users to scrap data from various sources of football data. One of example of portal with data to be used in this work is Transfermarkt [5] - German-based website owned by Axel Springer SE that has footballing information, such as scores, results, statistics, transfer news, and fixtures. Another useful website from which we can scrap the data easily using `worldfootballR` package is FBRef [6] - a website of football stats, history, and scores, for 100+ men and women club and national team competitions.

1.3 Data import and preparation

Dataset that will be the main object for analyses was retrieved using package `worldfootballR`, presented in the previous section. It keeps stats for all players that have been playing in teams of so-called 'big-5' [7] of European football - German, English, Spanish, French and Italian

leagues for the season 2022/2023. Firstly, data from Transfermarkt containing basic info of players and their market values were extracted with following code:

```
countries_for_tm <- c('England','Spain','Germany','France','Italy')
year_start <- 2022
year_end <- 2023
player_values <- tm_player_market_values(country_name = countries_for_tm,
                                          start_year = year_start)
```

Then, to get detailed statistics of each player in the observed dataset, applying of functions to retrieve data from FBRef resource is needed. Because individual's data in that portal are separated into different sections based on type of stat observed, there were selected some specific stat types, that are needed in the final dataset:

- Standard — basic stats of a player, covering his playing time(matches and minutes played), performance (goals/assists, yellow/red cards), and standard info , such as age, position, team that he plays for, etc.
- Shooting — detailed data on player's shooting abilities, e.g. overall number of shots, how many of them were on target, average shot distance.
- Passing — overview of player's amount of passes done and their accuracy, with dividing all passes completed into 3 groups by its distance.
- GCA (Goal-creating actions) — statistics, showing how often player's involvement in game lead to shots and goals.
- Defensive — players stats in defensive actions, covering their participation in duels, and how often and how successful they were tackles, interceptions, and blocks.
- Possession — this type of stats is aimed to describe player's involvement in actions of touching, receiving and carrying ball on the field.
- Goalkeeping — specific stat type only for goalkeepers and their specific actions, such as save percentage, number of matches finished with clean sheet(0 goals against).
- Miscellaneous — some other stats not included in any type above, e.g. penalty kicks won/conceded, amount of times player fouled/has been fouled, etc.

```
fbref_stats <- c("standard", "shooting", "passing", "gca", "defense",
               "possession","misc", "keepers")
for (stat in fbref_stats){
  path <- paste('./data/',stat,'_stats.csv',sep = '')
  stat_datset <- fb_big5_advanced_season_stats(
    season_end_year = year_end,
    stat_type = stat,
    team_or_player = 'player'
  )
  write.csv(stat_datset,file = path,row.names = F)
```

```
}
```

For the next steps of data preparation, function 'player_dictionary_mapping()' is used, as it allows user to merge player datasets extracted from FBRef and Transfermarkt, using columns 'UrlFBref' and 'UrlTmarkt':

```
> player_dict <- player_dictionary_mapping()
> colnames(player_dict)
[1] "PlayerFBref" "UrlFBref"    "UrlTmarkt"   "TmPos"
```

Then, as all raw data are extracted from sources, further data processing is needed. Firstly, all dataframes were combined into one with basic info of players and relevant columns added for each stat type observed. After that, that main dataframe was modified to filter out all players that didn't play at least 15 matches, as their stats could be biased by low number of matches played, and will not show player's performance on a longer distance run. Then, for better interpretation, some variables needed to be converted. As observations in combined dataset are players from teams in one of five best leagues in Europe, difference in total number of matches played across these leagues should be considered - while in German competition 18 teams play in total 34 games each, there are 38 matches played in England, Italy, France and Spain. Therefore, all variables, that show values of some stat as a total amount, were converted into 'stats-per-game' type, so we can see that player who has scored 10 goals in 20 games is comparable to the one with 15 goals scored in 30 matches. Also, there are some variables in the dataset, that are representing distance, and it is measured in yards in source, so all these values were converted into meters based on the rate $1yd = 0.9144m$. Last filter that was applied is to exclude players, that have played for more than one team during that season, in order to look only at individuals that played in a consistent system whole time, as their specific stats could be affected by moving into a team with completely different playing style.

1.4 Data dictionary

After all necessary steps of data preparation is done, there is a final dataset that will be a base for analysis in next chapters. It contains 1414 rows, each representing one football player from one of the 98 team across 5 different European leagues, and total of 56 columns with some characteristic of a player. More detailed description of each column is presented in a table below:

Column Name	Description
player_name	Full name of player
comp_name	Name of competition in which player's team participates

squad	Player's team name
player_position	Position most commonly played by player
player_age	Player's age
player_market_value_euro	Player's market value in euro based on Transfermarkt valuation
PlayerFBref	Column added in technical purposes to connect data between Transfermarkt and FBRef
player_nationality	Nationality of the player
UrlFBref	Column added in technical purposes to connect data between Transfermarkt and FBRef
MP_Playing	Matches played by the player
SCA_SCA	Shot-Creating Actions per game The two offensive actions directly leading to a shot, such as passes, take-ons and drawing fouls.
GCA_GCA	Goal-Creating Actions per game The two offensive actions directly leading to a goal, such as passes, take-ons and drawing fouls
PassLive_GCA	Completed live-ball passes that lead to a goal per game
PassLive_SCA	Completed live-ball passes that lead to a shot attempt
SoT_Standard	Shots per game - does not include penalty kicks
SoT_percent_Standard	Percentage of shots that are on target
npvG_Expected	Non-Penalty Expected Goals per game
Gls_Standard	Goals scored per game
G_minus_xG_Expected	Goals minus Expected Goals per game
npvG_per_Sh_Expected	Non-Penalty Expected Goals per shot
Dist_Standard	Average distance, in meters, from goal of all shots taken
Cmp_percent_Total	Pass Completion Percentage
KP	Passes that directly lead to a shot (assisted shots) per game
xAG	Assists minus Expected Goals Assisted per game
Final_Third	Completed passes that enter the 1/3 of the pitch closest to the goal per game
PPA	Completed passes into the penalty area per game
PrgP	Progressive Passes Completed passes that move the ball towards the opponent's goal line at least 10 yards from its furthest point in the last six passes, or any completed pass into the penalty area per game. Excludes passes from the defending 40% of the pitch
PrgDist_Total	Progressive Distance Total distance, in meters, that completed passes have traveled towards the opponent's goal.
Ast	Assists per game

Cmp_percent_Long	Pass Completion Percentage Passes longer than 30 yards(27.432 meters)
Cmp_percent_Short	Pass Completion Percentage Passes between 5 and 15 yards(4.572 and 13.716 meters)
Cmp_percent_Medium	Pass Completion Percentage Passes between 15 and 30 yards(13.716 and 27.432 meters)
CrsPA	Completed crosses into the penalty area per game
Tkl_percent_Challenges	Dribblers tackled divided by number of attempts to challenge an opposing dribbler
Tkl_Tackles	Number of players tackled per game
Clr	Clearances per game
Blocks_Blocks	Number of times blocking the ball by standing in its path per game
Int	Interceptions per game
TklW_Tackles	Tackles in which the tackler's team won possession of the ball per game
Pass_Blocks	Number of times blocking a pass by standing in its path per game
Sh_Blocks	Number of times blocking a shot by standing in its path per game
Touches_Touches	Number of times a player touched the ball per game. Note: Receiving a pass, then dribbling, then sending a pass counts as one touch
Succ_percent_Take	Percentage of Take-Ons Completed Successfully Unsuccessful take-ons include attempts where the dribbler retained possession but was unable to get past the defender
TotDist_Carries	Total distance, in meters, a player moved the ball while controlling it with their feet, in any direction per game
PrgC_Carries	Carries that move the ball towards the opponent's goal line at least 9.144 meters from its furthest point in the last six passes, or any carry into the penalty area per game. Excludes carries which end in the defending 50% of the pitch
Final_Third_Carries	Carries that enter the 1/3 of the pitch closest to the goal per game
Rec_Receiving	Number of times a player successfully received a pass per game
CPA_Carries	Carries into the penalty area per game
PrgR_Receiving	Progressive Passes Received Completed passes that move the ball towards the opponent's goal line at least 9.144 meters from its furthest point in the last six passes, or any completed pass into the penalty area per game. Excludes passes from the defending 40% of the pitch
Save_percent	Goalkeeper stat only. Save Percentage

CS_percent	Goalkeeper stat only. Percentage of matches that result in clean sheets.
Save_percent_Penalty	Goalkeeper stat only. Penalty Kick Goals Against/Penalty Kick Attempts
GA	Goals Against per game
Won_Aerial	Aerials Won per game
Recov	Number of loose balls recovered per game
Fld	Fouls Drawn per game

Table 1.1: Data dictionary

1.5 Values standardization

One thing should be taken into account on this exact dataset. Firstly, values across different columns are presented in different measures, as there are some expressed in absolutes, while another represent percentage. Moreover, even for variables, their values are absolutes, ranges of values can vary a lot. For example, descriptive statistics for column *Touches* tell us, that mean value is 40.42, with standard deviation equal to 16.19:

```
> describe(midfields$Touches_Touches)
  vars    n  mean    sd median trimmed  mad  min   max range skew
X1    1 410 40.42 16.19  38.89   39.42 17.08 9.69 95.97 86.28 0.61
```

Meanwhile, column *FinalThird_Carries* represents actions, that happen during the match much more rarely, therefore having values with much lower variance and range:

```
> describe(midfields$Final_Third_Carries)
  vars    n mean    sd median trimmed mad  min   max range skew
X1    1 410 0.95 0.57   0.81   0.88 0.5 0.05 3.23   3.18 1.12
```

In order not to let these differences have an impact while calculating distances between objects, all stats were transformed into values acting as values from standardized normal distribution, using column means and standard deviations: [8]

$$z = \frac{x - \hat{x}}{\sigma} \quad (1.1)$$

where x - original value in dataset for each observation in the following column, $\hat{x}$ - mean value of column, σ - column standard deviation, z - new value for observation in following column after scaling. After dataset was scaled, differences in absolute values wouldn't affect distances calculations in favor of one variable against another.

1.6 Position specificity

After all data cleaning, preparation and definition processes are done, there is only one thing left to mention before start of analyzing given dataset. As variables cover stats of a lot of different types, it is obvious that players with different roles are expected to have non-identical values for most of the stats, without dependence on player's individual style of play. That's why while comparing players similarities, it is relevant to do so only after separating individuals into groups based on their position on the field. Original dataset will be divided into 6 subsets:

1. Goalkeepers — the most different group from every other. Final frontier between opponent's attacker and net, should be successful in saves.
2. Wing-Backs — defenders, that play on left/right wing, expected to be good as in defensive actions, as in attack support.
3. Centre-Backs — the most important players near the own net before goalkeeper, expected to won a lot of tackles, aerial duels, intercept and block ball a lot.
4. Midfielders — players with a lot of different functions on the pitch, ones that control pace and attitude of how team plays. Some could be more creative, trying to give their forwards as much chances to score as possible, another midfielders are inversely aimed to destroy opponent's attacking plans and begin counterattacks.
5. Wingers — usually the most dangerous players for the opponent, mostly fast and agile ones that either help midfielders in creating chances for striker, or try to dribble past opponent's defense in order to finish attacks by themselves
6. Forwards — most commonly, have the least defensive responsibilities within team, and always aimed on opponent's net while waiting for chances created for them by midfield and wingers to score the goal.

Now, as all data manipulations are described and applied, dataset is ready to be used in analysis. Due to the high amount of variables across the dataset, next step to be considered is an applying of PCA (principal component analysis) as a dimension reduction technique.

2. Principal Component Analysis

This chapter provides an overview of one of the most used technique for dimension reduction in data. [9] After PCA is applied, and principal components replace original variables, analyses could be done with lower complexity on less columns, keeping meanwhile most part of the variance from primary source. Moreover, datasets with lower number of variables are easier for exploration and visualization. Main concept at the core of PCA algorithms is calculating eigenvalues from covariance matrix in order to identify, what variables or variable combination could give as much information as possible in as less dimensions as possible.

2.1 Covariance matrix

First step needed to be done to prepare dataset for principal component analysis is a construction of covariance matrix. Covariance is a measure of relationship between two variables, that indicates, whether these variables tend to move in same or inverse direction - in other words, if higher values of one dataset column coincide with higher values of the second one. Formula to calculate covariance of two variables is following: [10]

$$cov(x,y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{n - 1} \quad (2.1)$$

where x_i is i -th value of variable x , $\bar{x}$ is a sample mean of variable x , y_i is i -th value of variable y , $\bar{y}$ is a sample mean for y , n is a number of observations in sample.

Covariance matrix is then a symmetric matrix $n \times n$, where n is a number of variables in dataset. Each element $n_{(i,j)}$ in that matrix represents covariance between i -th and j -th variables, so basically, elements $n_{(i,i)}$ on main diagonal are calculated, referring to the formula 2.1, as a variance of i -th variable. For example, if original dataset has 3 columns, represented as x,y,z , then covariance matrix would look like:

$$\begin{bmatrix} cov(x,x) & cov(x,y) & cov(x,z) \\ cov(y,x) & cov(y,y) & cov(y,z) \\ cov(z,x) & cov(z,y) & cov(z,z) \end{bmatrix} \quad (2.2)$$

Or, knowing, that $cov(x,x) = var(x)$, structure of covariance matrix can be rewritten as:

$$\begin{bmatrix} var(x) & cov(x,y) & cov(x,z) \\ cov(y,x) & var(y) & cov(y,z) \\ cov(z,x) & cov(z,y) & var(z) \end{bmatrix} \quad (2.3)$$

2.2 Eigenvalues and eigenvectors

After covariance matrix is built, next step of a PCA performing process is a calculation of eigenvectors and eigenvalues of this matrix. Definitions of these concepts could vary, however the most common one is that for the matrix A there is an eigenvector x , that satisfies following:

$$Ax = \lambda x \quad (2.4)$$

where λ is a scalar known as eigenvalue [11]. As it can be seen from 2.4, number of eigenvalues and eigenvectors is equal to the number of dimensions in data. Therefore, for dataset considered in this thesis, having 41 variables of different footballers stats, there is a covariance matrix 41×41 , with following number of eigenvalues and eigenvectors. The higher is the eigenvalue of a particular eigenvector, the higher is amount of variance from original dataset that is explained by corresponding principal component built by that eigenvector.

2.3 Principal components

Construction of principal component using retrieved eigenvectors from a covariance matrix is pretty simple. Sum of eigenvalues in dataset represent total variance, so each eigenvalue divided by their total shows percentage of variance explained by that component. Components are calculated as linear combinations of original variables in a way that there is no correlation between output components, so apart from dimension reduction, PCA also could be applied to get rid of multicollinearity. With matrix notation, knowing that X is a data matrix, $X = P\Delta Q^T$, where P - matrix of left singular vectors, Q represents right singular vectors, and Δ is a diagonal matrix of singular values, following formula is used to retrieve original data projections on directions of principal components:[12]

$$Y = Q^T X \quad (2.5)$$

As was mentioned above, eigenvalue represents amount of variance explained by principal component that assigned to following eigenvector. Based on explained variance, it is then a task to decide how many components to keep in data in order to reduce amount of dimensions as much as possible while saving most part of variance from source. One of the most traditional approach is to keep those components, eigenvalue of those is higher than the average across given vector of eigenvalues.[12] When covariance PCA is applied, that rule can be simplified to keep components with eigenvalues higher than 1, as dataset was normalized and sum of variances of all columns is equal to their number:

$$\lambda_i > 1/n \sum_{i=1}^n \lambda_i \implies \lambda_i > 1 \quad (2.6)$$

where n is a number of eigenvalues, and λ_i is an eigenvalue for i -th eigenvector.

2.4 PCA outputs

There are several characteristics, that can be retrieved from the result of principal component analysis application. Each of them can be useful to describe relationships between newly created dimensions and original variables or observations in data.

2.4.1 Variable to component correlation

In order to get information, how each of variable is explained by components, correlation matrix $n \times k$ can be created, where n is a number of original attributes in dataset and k is an amount of components kept using 2.6 approach. Each element of this matrix C is presented as: [13]

$$c_{i,j} = \frac{n\Sigma X_i Y_j - \Sigma X_i \Sigma Y_j}{\sqrt{[n\Sigma X_i^2 - (\Sigma X_i)^2][n\Sigma Y_j^2 - (\Sigma Y_j)^2]}} \quad (2.7)$$

where X_i is an i -th variable and Y_j is a j -th component.

2.4.2 Variables contributions

Correlations between variables and components are good to show importance and direction of original attribute to the calculated component. However, it is not possible to calculate overall importance over these values, as it could be a strong negative correlation which is not interpretable in terms of 'percent of contribution'. That's why these correlations should be squared and then each variable's contribution is calculated as:

$$contrib(x_i, y_j) = \frac{cor(x_i, y_j)^2}{\Sigma_{k=1}^n cor(x_k, y_j)^2} \quad (2.8)$$

where x_i is an i -th variable, y_j is a j -th principal component, and it is also valid that $\Sigma_{k=1}^n cor(x_k, y_j)^2$ is equal to eigenvalue of particular j -th eigenvector. [12]

2.4.3 Variance explained

As was already mentioned multiple times, eigenvalues are the values for which should be looked when calculating original variance explained by principal components. As datasets can be different and so can ranges of values for different variables, universal calculation for percentage of variance explained is calculated as following:

$$\%S_{explainedbyi} = \frac{\lambda_i}{\Sigma_{k=1}^n \lambda_k} * 100 \quad (2.9)$$

3. Measurement of similarity between objects

All the statistical methods described and applied in following chapters would be related to similarity measurement between observations in dataset. Metric that is used in this thesis was chosen to be Euclidean similarity, which is a method derived from *dissimilarity* measure, Euclidean distance.

3.1 Similarity metrics

When two sets of values A and B are given, similarity metrics are used in order to quantify dependency between these sets.[14] Similarities calculated for observations coordinates on principal components therefore would show, how close objects are to each other on the principal component space. Main prerequisites for each similarity metric are following:

1. Finite range – $S(A,B) \leq x$, where x is a some finite number, representing the highest possible similarity between sequences
2. Reflexivity – $S(A,B) = x$ only when $A = B$
3. Symmetry – $S(A,B) = S(B,A)$, as order of value sets does not matter in similarity calculation

There are lots of different methods to calculate a similarity between objects, each of them can be used in different type of problem. Some of the most used techniques are:

- Cosine similarity
- Jaccard similarity
- Pearson correlation coefficient

3.2 Dissimilarity metrics

Analogous to similarity metrics, dissimilarity ones measure same thing but from the opposite perspective, specifically independence between sequences A and B instead of dependency. Same as for similarity measures, dissimilarity have their own assumptions:

1. Non-negative values – $D(A,B) \geq 0$ for each A and B
2. Reflexivity – $D(A,B) = 0$ only when $A = B$
3. Symmetry – $D(A,B) = D(B,A)$

One of the most used technique to measure a dissimilarity between sets of values is a cal-

culatation of distance between objects. There are multiple approaches to distance measure, differing in a way how metric is calculated. Some of the most used distances are:

- Euclidean distance
- Chebychev distance
- Manhattan distance
- Minkowski distance

In the context of this thesis, Euclidean distance was chosen as an appropriate metric to measure distance between observations

3.3 Euclidean similarity matrix

Consider a $p \times n$ data matrix Z composed of n columns, each p elements long. The Euclidean distance between vectors z_i and z_j is then: [15]

$$d_{ij} = [(z_i - z_j)^T(z_i - z_j)]^{1/2} \quad (3.1)$$

When computed for all i, j , and arranged in a matrix in an order identical to that obtained from matrix cross products, this process results in a symmetric dissimilarity matrix of Euclidean distances, D . For n cases, D will be $n \times n$. This matrix has zeros along the main diagonal (because the distance between a vector and itself is zero) and has units identical to the input data. The difference between dissimilarity and similarity is orientation, so for a Euclidean dissimilarity metric, large values indicate a large Euclidean distance, whereas for a Euclidean similarity metric, large values indicate a small Euclidean distance. To create a similarity matrix, some simple operations must be applied to D . Define d_{max} as a maximum value in D . Using this d_{max} normalization is done such that no element in the newly created $\hat{D}$ is greater than 1

$$\hat{D} = (1/d_{max})D \quad (3.2)$$

Then, let A be the matrix with all elements equal to 1. Matrix S , that is considered to be similarity matrix based on Euclidean distance, is then defined as: [15]

$$S = A - \hat{D} \quad (3.3)$$

Now, matrix S , which mimics correlation matrix, presents normalized values of similarities between n observations, and has ones in the main diagonal.

4. Cluster analysis

Cluster analysis is a multivariate statistical method that is based on assembling objects into groups in the way, so members of each group, that is called a cluster, are more similar between each other than they are to another clusters objects. This technique is traditionally being referred as unsupervised pattern recognition, because mostly there are no any predefined classes in the data that being clustered, in contrast to, for example, discriminant analysis, where its results are being validated on original categories from dataset. [16] This chapter would provide an overview of different approaches to cluster analysis, and more detailed description of a certain method, k-means clustering, with its further application on the subset of midfielders from original dataset of football players and their stats.

4.1 Hierarchical clustering

As there is a lot of clustering algorithms, there was a need to categorize them into bigger groups as well. Mainly, these techniques are being referred either as hierarchical, or partitional. Big difference between methods is in their approach to number of clusters determination in the output. Hierarchical clustering algorithms do not require any predefined amount of clusters, as each of them starts from either one cluster containing all individual observations from data, and gradually separating these observations from each other into n clusters, where n is equal to number of records in dataset (*divisive hierarchical clustering*), or, vice versa, clusters each having only one element in them, gradually merging these clusters into a single one containing all dataset individuals (*agglomerative hierarchical clustering*). [17] Due to computation complexity, divisive methods are not as used as agglomerative ones. Sequence of steps done while performing hierarchical clustering is often being visualized as a binary tree display, which is being referred to as 'dendrogram'. Main concept used in partitioning objects on hierarchical clustering is a presented in previous chapter dissimilarity measure. Based on how distance metrics are used while decision making how to combine objects into clusters, there could be different procedures used: [17]

- Single linkage – method of finding nearest neighbor. When objects x and y are grouped in a cluster A , new cluster would be created combining cluster A and an object z with minimum $D_{A,z}$ calculated using formula

$$D_{A,z} = \min(D_{x,z}, D_{y,z}) \quad (4.1)$$

- Complete linkage – method of finding the furthest neighbor. When objects x and y are grouped in a cluster A , new cluster would be created combining cluster A and an object z with minimum $D_{A,z}$ calculated using formula

$$D_{A,z} = \max(D_{x,z}, D_{y,z}) \quad (4.2)$$

- Average linkage – method of mean distance calculation between clusters. Two clusters A and B would be merged into cluster C , if they have the lowest value of $D_{A,B}$ calculated using formula

$$D_{A,B} = S/(n_A * n_B) \quad (4.3)$$

where S is a sum of all pairwise distances between clusters, and n represents length of a following cluster.

Despite advantages of hierarchical clustering, when working with larger dataset, it could be quite challenging as from memory, as from time perspective to process these algorithms. Therefore, clustering methods, where number of classes is predefined, are also being used pretty often.

4.2 Partitional clustering, k-means algorithm

As mentioned above, partitional clustering techniques assume that number of clusters, to which we want to separate original dataset, is defined before. [18] These clustering methods are mainly based on iterative allocation/reallocation objects until reaching convergence. That allocation of observations is determined by comparing distance between object of a cluster and its center point called centroid, to centroids of other clusters, whether that particular data point could be reassigned to another group. One of the simplest and therefore most widely used partitional clustering technique is a K-Means algorithm. Main goal to achieve while applying that method is to minimize within-cluster sum of squares:

$$\sum_{i=1}^k \sum_{j=1}^n (x_j - M)_i^2 \quad (4.4)$$

where k is a predefined number of clusters, n is a number of objects in dataset, and $(x_j - M)_i^2$ is a distance between object and a cluster centroid. Simplified algorithm of K-Means application is following: [19]

1. k is chosen as a number of clusters into which data points are aggregated
2. k initial group centroids are chosen (randomly or with any prior information given). As centroid is meant to be an average of all the points in the cluster, it would be a vector with same number of elements as the number of dimension in data.
3. For each object in original data, cluster is assigned based on the nearest initial centroid.
4. After objects are all assigned to some cluster, centroids should be recalculated as means of each dimension within a cluster.
5. Steps 3 and 4 then are needed to be repeated until centroids aren't being changed

Simplicity of that algorithm and its interpretation belongs to main advantages of K-Means clustering, along with method being faster from the side of computation than hierarchical approach when performed on larger sets of variables, and providing tighter clusters as well. There are, however, some disadvantages, that may include, for example, fact that K-Means does not actually identify data outliers, problems with handling data of different size and

density, and as a most obvious one is prior definition of number of groups, that could lead to erroneous interpretation of patterns in data. Last issue could, nevertheless, be handled with a number of techniques, helping to determine optimal number of clusters based on data.

4.3 Optimal number of clusters determination

There are different approaches on how to define the optimal number of cluster for the given dataset, all being different by their complexity and calculation time cost. [19]

4.3.1 Rule of thumb

In case of work with some basic application of cluster analysis, the most primitive rule that can determine number of clusters to categorize objects into is a so-called rule of thumb. By this rule, dataset with n observation can be divided into k clusters, where k defined as:

$$k = \lfloor \sqrt{\frac{n}{2}} \rfloor \quad (4.5)$$

This method is pretty simple, the only value that is considered is a size of dataset. However, rule of thumb application is efficient in case if objects in dataset could actually be divided equally into such high number of clusters and at the same time different cluster centroids differ from each other, which can't be assured by method that does not take into account specifics of dataset. To get the optimal number of clusters, which would be more appropriate in terms of a concrete clustering task, there are various methods used, their essence is in running the task multiple times with different number of groups, and then validation of the output by keeping value for which clustering gave the best result.

4.3.2 Elbow method

Elbow criterion is one of the most popular method helping to define appropriate number of clusters for the given dataset. Main advantage is in its easier interpretation, as that criterion is based on visualization of the retrieved within-cluster sum of squares by number of clusters defined, and finding an 'elbow' point on this graph, which determines, on which step of the analysis WCSS not being decreased significantly. For example, on the graph below there can be seen, that based on sum of squares value drop, 4 clusters can be considered as recommended number based on the elbow point.

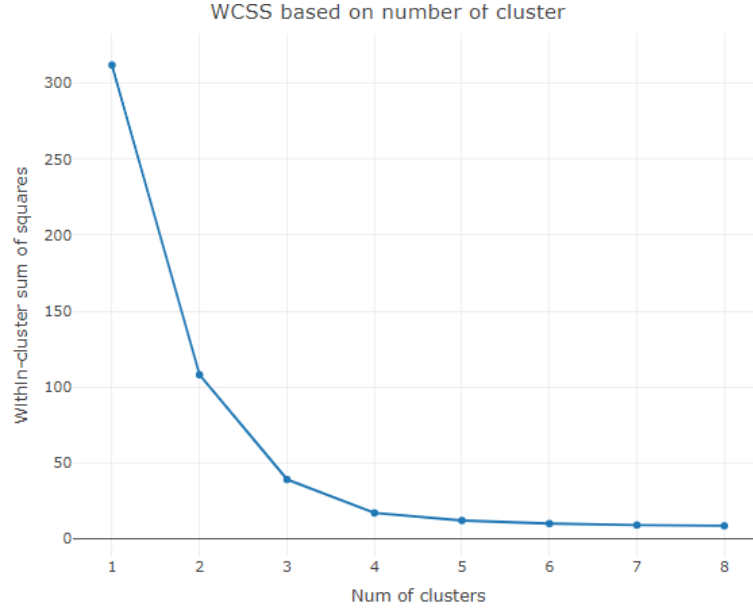


Figure 4.1: Example of cluster analysis viz to determine 'elbow' point. Own processing with R

4.3.3 Silhouette method

Another approach, on how to determine number of clusters that would help to aggregate dataset objects into groups where objects are as most similar between each other inside the group, as they are different from the objects of another cluster, is a calculation of silhouette coefficient. Same as Elbow method, silhouette score helps to evaluate one particular option of clustering of the dataset in order to compare its result to another options applied on the dataset. Base of that method is evaluation of distance of each object in data to its 'neighbors' from the same cluster where they belong to, against distance of same object to observations from the nearest cluster to its own. So, let x be the dataset object, A be the cluster to which x belongs with m elements, and B is the nearest cluster to A (centroids used to obtain that information) with k elements. Then, after calculation average distances between x and objects from both these clusters:

$$a_x = \frac{\sum_{y \in A, y \neq x} d(x, y)}{m - 1} \quad (4.6)$$

$$b_x = \frac{\sum_{y \in B} d(x, y)}{k} \quad (4.7)$$

silhouette score can be obtained as following:

$$s_x = \frac{b_x - a_x}{a_x} \quad (4.8)$$

Therefore, it is seen, that when $a_x > b_x$, value of silhouette score is negative, and it indicates that observation was assigned to an incorrect cluster, while positive values for that coefficient tell that assignment was correct, and higher the value, more similar object is to the center of its own cluster than to objects from the nearest cluster. Values near or equal to 0 indicate

that clusters are overlapping. Using silhouette coefficients, calculated for each individual in the dataset, it is possible to evaluate overall performance of clustering by averaging scores:

$$S = \frac{\sum_{i=1}^n s_i}{n} \quad (4.9)$$

That average silhouette score would obviously have the similar interpretation of clustering - the closer value is to 1, the more objects from dataset were classified to the right cluster, so the optimal number of clusters is the one which would have the highest mean silhouette coefficient.

4.3.4 Midfielders dataset optimal number of clusters

Now, to determine suitable number of clusters in the given dataset of football players, iteration over different values (from 3 to 8) was done with calculating both WCSS and Silhouette score for the given clustering output, as well as for the number of clusters derived from 4.5, which in the following case equals to $\lfloor \sqrt{\frac{410}{2}} \rfloor = 14$. Summary of clustering done is following:

# of clusters	Silhouette	WCSS
3	0.244	7568.580
4	0.184	6825.948
5	0.168	6205.727
6	0.170	5768.558
7	0.159	5485.663
8	0.149	5238.686
14 (by 'rule of thumb')	0.141	4186.485

Table 4.1: Comparison of WCSS and average silhouette scores after running k-means algorithm with different number of clusters

As can be seen from the table, average silhouette coefficient significantly drops when increasing number of clusters from 3 to 4, and that decrease is not very high for any further addition (with even 6 clusters model having more preferable result than with 5). To validate result by elbow method, either differences between WCSS can be calculated, or, for visual interpretation, line chart could be built:

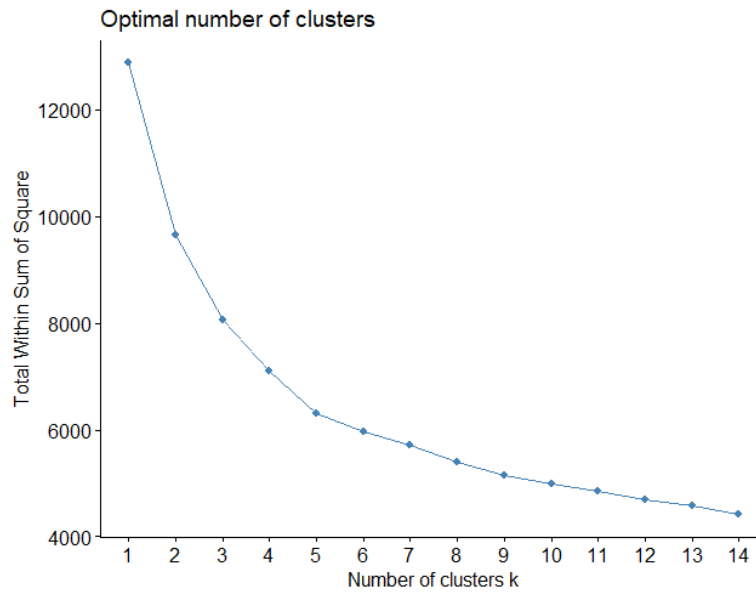


Figure 4.2: Chart to determine 'elbow point' for optimal number of clusters in midfielders dataset. Own processing with R

Same as with silhouette score, 3 can be considered as a suitable number of clusters to split midfielders from original data into.

5. Player replacement model

All the previous chapters have provided theoretical background on various metrics and statistical methods that can be used to study similarities between observations in datasets with a higher number of original dimensions. The next step is to move from theory to practice and demonstrate how these concepts can be applied in real-world scenarios. Given a dataset of football players, techniques that describe patterns in the data and relationships between observations could be useful in building a recommendation model. This model would use the analysis output to identify the best replacement candidates for any player in the dataset. This model can either be used by football clubs to directly find a replacement for their player, or to find an alternative for a player they may want but can't afford, as data from Transfermarkt allow to tune this model by filtering sportsmen by their price. Reference player was chosen as a player replacement is being looked for – centre-back Kim Min-jae, who left Napoli SSC after a very successful season as for his team, as for him individually, therefore Napoli board who benefited by more than €30 million, after buying him for €18.05m and selling to German grand Bayern for €50m, is now at the market to find a new center-back, who is able to replace Kim, and would get similar stats in the team as Kim had last season.

5.1 Tuning model - age and price filters

In order to assume, how would the 'perfect' player for Kim's replacement look from the point of his age, and how much money would they afford on a new signing, it could be useful to use historical data of Napoli transfer activity. Retrieving Transfermarkt records of players arrived to club in last 5 years, descriptive statistics for transfer fees and ages of new signings were calculated as following:

Statistic	Player's age	Transfer fee
Minimum	19	0
First quartile	22.25	753 750
Median	24	10 025 000
Mean	25	12 981 324
Third quartile	26.75	17 962 500
Maximum	35	75 000 000

Table 5.1: Descriptive statistics of transfer fee and age columns in dataset of Napoli arrivals last 5 years

From the table above can be concluded, that Napoli tends to buy younger players (half of the signings were not older than 24 and 75% of them are younger than 27), paying for them around €13 million in average (only for 4 out of 34 signings Napoli paid more than €30

million). However, given the fact that player Napoli is looking replacement for was one of the most important and valuable in the squad, and the fact that club earned pretty lot on Kim's transfer to Bayern, upper boundary for market value and age can be increased a bit, as they can allow to spend more and may prefer more experienced player. As a result, realistic filters for age and value of new Napoli defender could be set on the level of 28 years and 30 million euros. Dataset, for which following filters have been applied, was then scaled in order to perform PCA and identify stats that contribute to overall variance more.

5.2 Central defenders PCA application

Using mentioned in chapter 2 common method of optimal dimension number definition (keep components with eigenvalue higher than 1), final number was set to 11. This dataset would keep 78% of variance from the original one with 41 stats, with 36.5% of that explained by first two components. There are 3 variables, that contributed the most across all 11 dimensions - 'Tkl_percent_Challenges', 'Blocks_Blocks', 'PassLive_GCA'. Higher overall contributions means that original variable is highly correlated (does not matter if positively or negatively) with multiple principal correlations. For example, the more blocks player had, the significantly higher would be his value for 3rd and 4th dimensions and meanwhile it would go opposite direction primarily for components 9 and 5. Live passes lead to shot attempt had a high positive influence on 1st and 6th components while lowering 5th and 10th dimensions values:

Dimension	PassLive_GCA	Tkl_percent_Challenges	Blocks_Blocks
comp. 1	0.503	0.032	0.147
comp. 2	0.001	0.132	0.202
comp. 3	-0.156	0.102	0.546
comp. 4	-0.094	0.27	0.539
comp. 5	-0.331	0.132	-0.119
comp. 6	0.452	-0.039	0.036
comp. 7	0.117	0.262	0.455
comp. 8	0.394	0.096	0.036
comp. 9	0.029	0.312	-0.233
comp. 10	-0.28	-0.135	0.056
comp. 11	0.04	-0.648	0.07

Table 5.2: Principal component loadings for three most contributed variables

Using loadings for all 41 stats from the original dataset, matrix 157×11 was created for all players using 2.5. Values in Kim Min-jae's row are following: $x = (6.388, -2.506, 0.694, -0.331, -0.181, -0.017, 1.306, -1.358, 1.942, -1.034, -1.451)$. Using variables loadings, his coordinates on components could be interpreted. Based on x values, Korean defender seems

to have plenty of overall touches per game with a lot of them being passes to partners (as well with higher accuracy). As for defensive actions, loadings for 3rd dimension could be helpful, describing $Clr = 0.685$ and $Won_Aerial = 0.614$ it is seen that Kim is good in aerial duels and overall clearances of the ball, however lacking in interceptions and amount of passes blocked. As for defender, Min-jae has a very solid stats of assist, meanwhile not being the best in shots on target, which can be expected from tall center-backs on corner kicks. From this principal components interpretation, it is now more clear what a portrait of 'perfect' replacement for Kim looks like, and similarity matrix that would be built further would be able to identify these candidates.

5.3 K-means clustering for defenders

Clustering techniques are very useful in order to identify patterns in data and create bigger groups of individuals who would differ from another by some attributes. After k-means algorithm was applied on defenders dataset, there were 3 clusters created with following centroids:

$$C_1 = (-2.905, 0.642, -0.078, -0.161, -0.095, -0.108, 0.034, 0.003, 0.066, 0.121, 0.042)$$

$$C_2 = (3.791, 2.007, -0.109, 0.158, -0.094, -0.11, 0.122, -0.024, 0.059, 0.227, 0.06)$$

$$C_3 = (0.609, -1.623, 0.126, 0.061, 0.134, 0.154, -0.094, 0.01, -0.089, -0.226, -0.069)$$

There is some information that is noticeable from cluster centroids. First cluster aggregates players whose average value on first component is the lowest (so they in average are worse in progressive passes and passes made on final third, but better in Goals-xG difference), however their value is closest to 0 for second dimension (higher values on that component are affected by higher amount of PPA, lower by better pass accuracy), most part of other centroid vector elements are as well very close to 0, which, given that value for stats were standardized, states that by most of attributes players from this cluster perform close to mean performance.

Second cluster is better described as a group of sportsmen, their pass accuracy is not as good, which is however explained by higher amount of more risky passes done (as it is obvious that with higher risk accuracy could decrease however efficiency of these actions increase). Also, as per 4th component, second cluster individuals are the most efficient in blocks, meanwhile being worse in goal-creating actions (the only cluster with negative centroid for 8th dimension, that is highly correlated with variables GCA_GCA and $PassLive_GCA$).

As for represents of the third cluster, there are the most successful passers presented, as if coming back to original data, mean $Cmp_percent_Total$ value for these defenders is higher than 86%. Centroids of 5th and 6th dimensions, that positively correlate with various goal creating and scoring actions, states that footballers of 3rd clusters are the best in these stats across dataset. However, there is some lack of defensive actions noticed for this group, as it is the only cluster with centroids below zero for components 7 and 9, big contributors for

which are tackles and blocks stats.



Figure 5.1: Bar chart, showing amounts of players assigned to each cluster. Own processing with R

From 157 defenders, represented in the following dataset, second cluster is represented by almost twice lower number of footballers, than first and third (34 vs 58 and 65). It is explainable by mentioned above advantages and disadvantages of each group – as defenders with tendency to play more risky are able to do it mostly in big clubs, which attacking style expects, that all players, including defenders, are involved in actions on the opposition side of field, as well as individuals from this clusters having the highest stats for most of defensive actions. Reference player, Kim Ming-jae is assigned to this second cluster, therefore it can be seen that his attributes are exceptional and differ from most part of defenders. Overall distribution of players by clusters and their stats can be visualized on the scatter plot. As axis, two stats, that were main contributors for two principal components (either with negative correlation to component or positive):

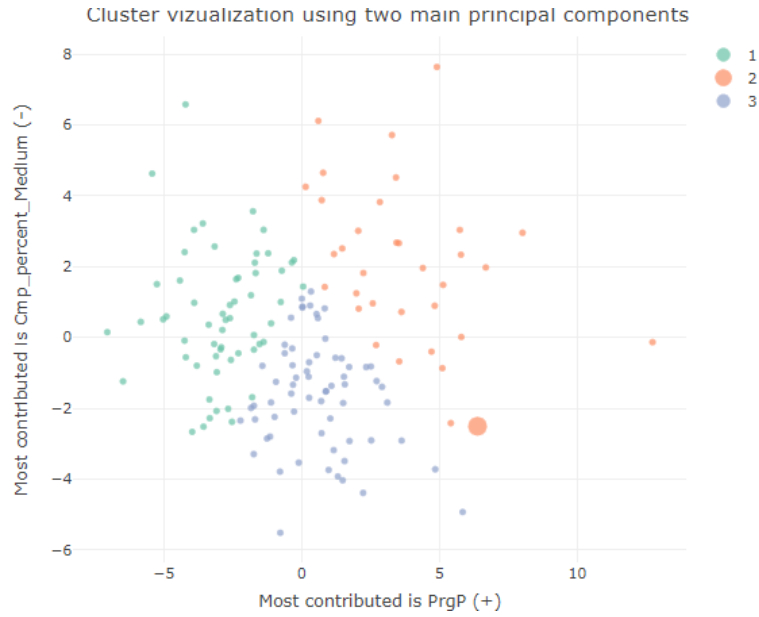


Figure 5.2: Scatter plot, revealing distribution of players by clusters. Kim Min-jae is highlighted as a data point with bigger size. Own processing with R

5.4 Similarity matrix - Ming-jae's replacement candidates

Finally, after PCA was applied for dimension reduction and cluster analysis performed to see common patterns in data, similarity matrix 157×157 can be constructed to get for each pair of observations in data a value that will represent how close they are to each other on the 11 principal component space. Similar to how it was done in chapter 3 for midfielders, firstly a dissimilarity matrix D was created, having as elements Euclidean distances between each pair of players. With $\max(D) = 20.524$ standing for distance between players Matteo Lovato (first cluster) and Facundo Medina (second cluster), values can be standardized and recalculated to similarities from distances, and similarity matrix S will be constructed:

	Min-jae Kim	Trevoh Chalobah	Joe Gomez	Davinson Sánchez
Min-jae Kim	1	0,608	0,697	0,462
Trevoh Chalobah	0,608	1	0,703	0,774
Joe Gomez	0,697	0,703	1	0,567
Davinson Sánchez	0,462	0,774	0,567	1

Table 5.3: Sample of 4 records from defenders similarity matrix

All values in similarity matrix are presented in range from 0 to 1, where 0 assigned to the highest distance in data (mentioned above case Lovato vs Medina), and 1 stands for diagonal elements, as it represents similarity of a player to himself. Using these objects, five most similar players to Kim Min-jae are retrieved:

Player	Cluster	Similarity
Edmond Tapsoba	2	0.791
Kevin Danso	2	0.747
Santiago Bueno	3	0.728
Yeray Álvarez	2	0.711
Adam Webster	3	0.709

Table 5.4: Top five replacements for Kim Min-jae by similarity

As output shows, three of the five most similar players to Kim belong to same cluster as a ex-Napoli defender. The best possible candidate offered from given dataset that has been filtered by conditions mentioned in 5.1 is Bayer Leverkusen centre-back, Edmond Tapsoba, with distance from Kim Min-jae equals to 4.3, converted to a 0.791 value of similarity between these two objects. In order to get a more detailed look on comparison of these two players, stats from original dataset with the highest contribution to principal components were put into radar chart visualization:

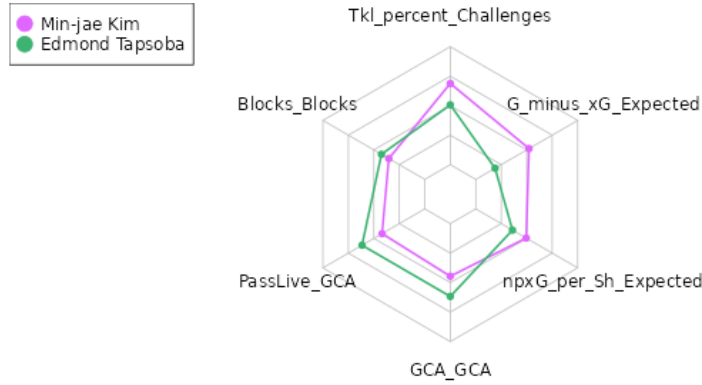


Figure 5.3: Kim Min-jae vs Edmond Tapsoba stats comparison on a radar plot. Own processing with R

As can be seen from the viz, players were pretty similar in amount of blocks done per game with a little Tapsoba's advantage on that stat, however his percentage of successful tackles is worse against Korean colleague (for that stat, Kim has value on level 86.2, which is the highest among the 157 defenders observed). Simultaneously, while Edmond Tapsoba can be considered as a more determined to creative actions, having better numbers in goal-creative actions and live-ball passes lead to a goal, Kim Min-jae was better in a scoring threat making, with great figures as against Tapsoba, as overall in comparison with all dataset observations in $G_minus_xG_Expected$ and $npG_per_Sh_Expected$. Referring to some other variables from original data, it is seen that these two players have similar values for further defensive stats, such as number of clearances per game (Kim 3.514, Tapsoba 3.303) or number of shots blocked (0.743 and 0.515). Both players were very successful in passing on short and medium distances, both having percentage above 90, while on long distances Tapsoba was less accurate. Overall, stats of footballers are pretty similar, and, given appropriate for Napoli market

value (Transfermarkt evaluates Leverkusen centre-back in €30 million), Edmond Tapsoba is the one that could be recommended as a replacement signing for Kim Min-jae.

If looking at other alternatives, RC Lens player Kevin Danso, who has a second highest similarity value to Korean defender, could be considered as a good option for replacement as well. Having some attributes appear more similar to Kim Min-jae, e.g. being better passer than Tapsoba, and closer to Kim in shot-creation actions (he has even the best value out of all three defenders for that stat on 1.108), he obviously has some disadvantages, as number and percentage of tackles won, or number of interceptions per game. Another point, that is worth mentioning, is a lower price per Transfermarkt projections against Tapsoba, as Kevin Danso is valued only at €22 million.

In case, if all filters applied would be reset, and five potential replacements are found on the entire population, here is how updated top-5 will look like comparing to 5.4:

Rank by similarity	Top with filters	Top on entire population
1	Edmond Tapsoba	Dante
2	Kevin Danso	Edmond Tapsoba
3	Santiago Bueno	William Saliba
4	Yeray Álvarez	Thiago Silva
5	Adam Webster	Pau Torres

Table 5.5: Top five replacements for Kim Min-jae - comparison between top with filters for age and price applied and without them

From the 5.5, there are 4 new players presented in the output without filters, with two of them (Dante and Thiago Silva) who were older than upper bound for age, and William Saliba and Pau Torres whose market value is higher than original filter for €30 million. Edmond Tapsoba, who was evaluated as the best replacement candidate in 5.4, remains in top-5 calculated on the entire population placed second.

5.5 Breakdown to different stat types

Although approach used above provides overview of players similarity studied on the level of all different stats available, it is not always the approach needed. Sometimes, aim of such analyses is to study patterns and look for similar players on the particular type of statistics. Original variables from the dataset could be divided into 4 groups (with fifth group for goalkeeping stats that can be relevant obviously only for keepers):

- Attacking
- Defensive
- Passing
- Possession

This approach allows to provide more detailed analysis on specific variables, as there would be less of them, so their influence and contribution to principal components expected to be more significant. For example, if studying defensive stats, there are four principal components created, with *Final_Third*, *Cmp_percent_Total*, *Ast* and *CrsPA* being highest contributors to each one of them.

Most correlated variable	Final_Third	Cmp_percent_Total	Ast	CrsPA
Correlation	0.838	0.897	0.684	0.433

Table 5.6: Overview of variables with highest correlation to each of principle components - only passing stats analyzed

To highlight significance on transferring from overall stats analysis to specific, top five replacements for Kim Min-jae by similarity value were found for passing and possession stat types:

Rank by similarity	Player (passing)	Player(possession)
1	Kevin Danso	Arthur Theate
2	Levi Colwill	Castello Lukeba
3	Edmond Tapsoba	Kevin Danso
4	Axel Disasi	Edmond Tapsoba
5	Clément Lenglet	Ko Itakura

Table 5.7: Top five replacements for Kim Ming-jae - by passing and possession stats

As can be seen, switching between different types of stat influences an output a lot, as only two players who belong to 'overall' top-5 from 5.4 remained in these tops for passing and possession, and even for them it differs a lot, as Edmond Tapsoba, who was considered as the most similar player to Kim by all 41 variables, has worse similarity values for both separated groups of stats than second-placed Kevin Danso (however, it clearly confirms, that, as was mentioned before, Danso's passing style is very similar to Min-jae's). Therefore, it is very important to determine before the start of analysis, if the task is in scrolling through all possible stats available, or specific game style preferred to be analyzed.

6. Shiny web application

In order to get the analysis output, that was described in the previous chapter, an R code environment was used. There was number of functions created and then applied on the data sources loaded in the environment and then multiple graphs and tables were shown as a result. However, when there is some business task given and it was decided to create some technical solution for this task, there is always a question, how this solution is going to be implemented into business processes? Using an interactive app instead of source code environments is not only efficient from the point of time saving, but also expands possible target audience of users. When some business requirement is given (in case of that thesis, it is a requirement to prepare a list of possible replacements for a given player), having an interactive environment with some user interface will allow people from business side use and get outputs from the solution developed once on a regular basis without need to request 'technical stuff' to run the task each time on their side and waiting for their outputs. Being more specific to the thesis theme, developing an app would allow users to have an interface, where they could pick any player from the dataset, choose any stat type and get analyses results without editing (and therefore without any prior R knowledge) source code.

As calculations and visualizations done in this thesis already were processed using an R programming language, its package Shiny was chosen as a tool to develop a web application for the 'player replacement model'. Main goal of the app is to allow run of all the described analyses on any player from the dataset without need to change anything in code each time new analysis is needed.

6.1 Shiny environment

Shiny, being a framework to R environment allowing web application building, was primarily need to satisfy needs of statisticians, data analysts, and data scientist, therefore it was developed in a way so applications can be created without prior knowledge of HTML, CSS, JavaScript or any other technologies used in web development. However, with all its user components that can be easily extended, Shiny is effective in tons of different use cases, becoming so popular that it even comes pre-installed in RStudio, most popular IDE used for R development. [20] Apart from providing outputs of complex models implementation to a non-technical audience, which is exactly the task to be done for the player replacement model, Shiny apps are also widely used to create interactive dashboards that can track important activities and indicators, or, for example, to create tools for teaching some statistics or data analysis basics.

6.2 Main Shiny concepts

Each Shiny app consists of two core modules – UI and server. UI (user interface) is a part of an app which defines appearance of the application, HTML web page users interact with, while server is responsible for how application processes work. Reactive programming used in Shiny is a set of functions, that allow application to automatically update outputs with inputs changed.

6.2.1 User interface module

When defining appearance of the application in UI part, *fluidPage()* is a core module, that is being base for all implemented modules and functions. Two main components inside *fluidPage()* are *titlePanel()* and *sidebarLayout()*, both allowing to enter functions that either read user input or show output to him, and differ on the position of the inputs/outputs on main window. Here is how different app components are located:

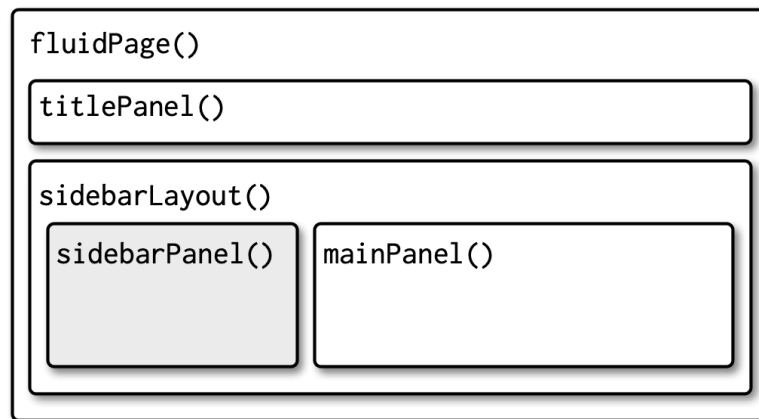


Figure 6.1: Shiny application structure - different modules with their location.[20]

In purposes of player replacement model, various functions suitable for UI component are implemented into app. Title is shown on the *titlePanel()*, while *sidebarLayout()* is divided into two panels - sidebar on the left, and main. Input selections and sliders added to the sidebar panel to ask user for a player he wants to find similar ones to, filters for age and market value needed to be applied and number of clusters to divide players into. Tick boxes are also implemented into user interface to give user a choice, which details of principal component analysis (variable correlation to component, variable contribution, eigenvalues of components) to visualize in the output. Action button is added at the end of side panel, to let user run the server functions based on all input info specified and process their result into the outputs.

Choose a player who you'd want to replace and select filters

Player position:
Centre Back

Select League
Serie A

Select Team
SSC Napoli

Select Player
Min-jae Kim

Type of stat:
Defensive

Player market value:
5,000,000 30,000,000 200,000,000

Maximum age
18 28 45

Number of possible replacements
1 5 10

Number of clusters to create
2 3 7

☒ Show PCA variable contributions
☐ Show original variable correlations to PCA dimensions
☐ Show PCA variance explained

Find players

Figure 6.2: Sidebar appearance of the Shiny app for player replacement model. Own screenshot from R Shiny app

These outputs are defined to be shown on the main panel of app, which is divided into four panes - 'Data Table', 'Best Replacement - PCA plot', 'PCA results' and 'Cluster visualization':

1. Data Table - shows a row from original data with stats of a reference player, and a table with n rows, where n is a number of potential replacements set by user, showing same stats for the most similar players to a reference one.
2. Best Replacement - PCA plot - radar chart, that visualizes comparison of chosen player to his replacement candidate (*selectInput()* function added to allow to choose to the radar chart any player from the n replacements in output) by the most contributed variables to principal components.
3. PCA results - based on number of boxes from sidebar ticked, provides from 0 to 3 tables, each representing different PCA result.
4. Cluster visualization - scatter plot, where all observations from dataset are colored by

the cluster they assigned to, and reference player is highlighted as data point with bigger size.

Player	comp_name	squad	player_position	player_age	player_market_value_euro	npxG_Expected	npxG_per_Sh_Ex
1 Min-jae Kim	Serie A	SSC Napoli	Centre-Back	26	60000000	0.029	

Showing 1 to 1 of 1 entries

Player	comp_name	squad	player_position	player_age	player_market_value_euro	PlayerFBref	player_nationality
1 Edmond Tapsoba	Bundesliga	Bayer 04 Leverkusen	Centre-Back	24	30000000	Edmond Tapsoba	Burkina Faso
2 Kevin Danso	Ligue 1	RC Lens	Centre-Back	25	22000000	Kevin Danso	Austria
3 Santiago Bueno	LaLiga	Girona FC	Centre-Back	24	8000000	Santiago Bueno	Uruguay
4 Yeray Alvarez	LaLiga	Athletic Bilbao	Centre-Back	28	15000000	Yeray Alvarez	Spain
5 Adam Webster	Premier League	Brighton & Hove Albion	Centre-Back	28	25000000	Adam Webster	England

Figure 6.3: Main panel appearance of the Shiny app for player replacement model. Own screenshot from R Shiny app

6.2.2 Server module

In order to transform input parameters into output tables and plots, that are seen to user and defined on UI module of application, R code chunks should be inserted into server module. To make code more clean, all the custom functions, that were created in the context of player replacement model, are saved in a separate R script, and that script is imported on the header of app script:

```
source('pca_for_app.R')
```

One of the most important concept, that is useful in server module of Shiny app, is a reactive expression. These functions allow to change output of application in case if input can be dynamically changed. As an example, in given application there are *selectInput()* functions added to firstly choose position, then league, then team and only at the end a player who replacements are looking for based on above picked options. In order to get correct list of items for the lower level, correct reactive expressions must be set for lists of items above. Firstly, list of players on specific position is added as a reactive block:

```
filtered_data <- reactive({
  positions[[input$position]]
})
```

Based on that data and league chosen, list of clubs is being retrieved reactively :

```
club_pick <- reactive({
  filtered_data_league <- filtered_data() %>% filter(comp_name == input$selected_league)
  return(unique(filtered_data_league$squad))
})
observe({
  updateSelectInput(session, 'selected_club', choices = club_pick())
})
```

And finally, based on a team picked, list of all players of that team on specified position is refreshed:

```
players_pick <- reactive({
  filtered_data_squad <- filtered_data() %>% filter(squad == input$selected_club)
  return(unique(filtered_data_squad$player_name))
})

observe({
  updateSelectInput(session, 'selected_player', choices = players_pick())
})
```

As there are differences between stat types, which are important for outfield players and goalkeepers, list of available stats based on position picked is also implemented as a reactive output:

```
chosen_stats <- reactive({
  available_stats[[input$position]]
})
```

No less important element of server module in Shiny is an *observeEvent()* function. When button is added into UI input, that function can specify, what specific tasks should be ran after clicking that button. In the given solution, this is the most critical part of the back-end part, as all the functions defined (PCA application, cluster analysis, building of similarity matrix and pulling n best similarities from it) are covered into *observeEvent(input\$run_analysis)* chunk. All the outputs, that are shown in panes of app main panel, are also defined inside *observeEvent()* element.

6.2.3 App script structure

After server and UI modules are developed, final step is merging two modules and calling to them in order to compile an app. Script **app.R** is created, where on header all needed packages and helping scripts are imported, then server and ui functions are defined, and following command is written on the last line:

```

library(shiny)
library(shinythemes)
library(DT)
source('pca_for_app.R')
ui <- fluidPage(
  titlePanel("..."),
  sidebarLayout(
    sidebarPanel(...),
    mainPanel(...)
  )
server <- function(input, output,session){...}
# Run the app ----
shinyApp(ui = ui, server = server)

```

6.3 Web applications publishing

Despite created R script gives an output as an application with user-friendly interface, storing and sharing it as an .R file still requires from all users to have R/RStudio installed locally and execute that script each time they would want to access an app. In case, that there are people in app target audience, that do not have any experience with R, it is much better solution to upload and share output of that script as a web page. Posit, company that developed RStudio, offers three possible ways of Shiny apps hosting: [21]

- shinyapps.io
- Shiny Server
- Posit Connect

Hosting service shinyapps.io is the easiest to implement app with, as it can be connected directly from RStudio, and as soon as any enhancement done to app script locally on developer's PC, he can republish app onto server. Service provides variety of different usage plans based on customer needs, starting with free plan, on which amount of application allowed and active hours per month are limited (up to 5 applications with 25 active hours monthly) to Professional, that would be suitable to business, that is planning to build a lot of internal processes on applications developed in R, with unlimited number of apps, personalized domain, and many other bonuses.

Application, that was developed for player replacement model, was published with private access into shinyapps.io and can be accessed through the following *link*.

Conclusion

The main goal of this thesis was to introduce a so-called 'player replacement model', that uses statistical methods to determine similarities between football players in the given dataset. Firstly, dataset was introduced alongside with techniques of web scraping implemented into worldfootballR package for R programming language, which helped to retrieve data from various online sources collecting football stats. Before application of any statistical methods, dataset should have been cleaned and preprocessed, therefore some data manipulations were done and the whole dataset was divided into six groups of footballers based on position.

After that, main theoretical concepts were introduced in the following three sections of thesis. Principal component analysis was explained from its core, covariance matrix and eigenvectors with eigenvalues calculated to that matrix. Euclidean similarity, obtained from Euclidean distance, one of the most used dissimilarity metrics, was chosen as an indicator of relationship between footballers to be used. Partitional clustering technique *k-means* with its comparison to hierarchical clustering and description of its properties was described.

With statistical methods introduced and explained, practice part of thesis then offered an algorithm that could find replacement candidates based on similarity matrix of football players. As a reference footballer, whose replacement has been looking for, Kim Ming-jae, centre-back who played in Napoli SC and left them last summer was chosen. Subset of central defenders was filtered for age and market value based on historical data of Italian team signings, and set for players not older than 28 with market value 30 000 000 euros or lower. PCA identified 11 components based on 41 original variables, that would keep most of information, with three variables 'Tkl_percent_Challenges', 'Blocks_Blocks', 'PassLive_GCA', contributed to principal components the most. Korean defender was assigned to one of the three clusters, its players are expected to pass riskier, having a good amount of blocks, with disadvantage in lack of goal-creating actions. Despite Edmond Tapsoba was found as the most similar player to Kim when analyzing all stats given, there was identified big difference in similarity analysis across different stat types chosen.

To make solution more universal and applicable without need to change source code every time new player, filter or stat type should be entered into model, Shiny application was developed with interactive user interface. Thanks to this feature, even users without knowledge of R programming would be able to get outputs of analyses this model constructed on.

Literature

- [1] M. Lewis, *Moneyball: The art of winning an unfair game*, WW Norton & Company, **2004**.
- [2] Analysisport, "What Can Data Do for a Football Club?" A Case Study of Brentford F.C. <https://analysisport.com/insights/what-can-data-do-for-a-football-club/>, Accessed: 2023-09-01.
- [3] S. De S Sirisuriya in Proceedings of 8th International Research Conference, KDU, Published November 2015, **2015**.
- [4] J. Zivkovic, worldfootballR: Extract and Clean World Football (Soccer) Data, R package version 0.6.5.0003, **2024**.
- [5] A. S. SE, Transfermarkt, <https://www.transfermarkt.world/>, Accessed: 2023-09-01.
- [6] FBRef, FBRef database, <https://fbref.com/en/>, Accessed: 2023-09-01.
- [7] Statista Research Department, Big Five football, <https://www.statista.com/topics/5909>, Accessed: 2023-09-01.
- [8] P. J. M. Ali, R. H. Faraj, E. Koya, P. J. M. Ali and R. H. Faraj, *Mach Learn Tech Rep*, **1**.
- [9] M. Richardson, URL: <http://people.maths.ox.ac.uk/richardsonm/SignalProcPCA.pdf> (last access: 3.5. 2013). Aleš Hladnik Dr. Ass. Prof. Chair of Information and Graphic Arts Technology Faculty of Natural Sciences and Engineering University of Ljubljana **2009**, 6, 4.
- [10] Nikolai Janakiev, Understanding the Covariance Matrix, <https://datascienceplus.com/understanding-the-covariance-matrix/>, Accessed: 2023-09-01.
- [11] H. Abdi, *Encyclopedia of measurement and statistics* **2007**, 304–308.
- [12] H. Abdi and L. J. Williams, *Wiley interdisciplinary reviews: computational statistics* **2010**, 2, 433–459.
- [13] A. G. Asuero, A. Sayago and A. González, *Critical reviews in analytical chemistry* **2006**, 36, 41–59.
- [14] A. A. Goshtasby and A. A. Goshtasby, *Image registration: Principles tools and methods* **2012**, 7–66.
- [15] K. L. Elmore and M. B. Richman, *Monthly weather review* **2001**, 129, 540–549.
- [16] S. Landau and I. C. Ster, *Á Á* **2010**, 11.
- [17] F. Murtagh and P. Contreras, *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **2017**, 7.
- [18] T. S. Madhulatha, *arXiv preprint arXiv:1205.1117* **2012**.
- [19] T. M. Kodinariya, P. R. Makwana, et al., *International Journal* **2013**, 1, 90–95.
- [20] H. Wickham, *Mastering shiny*, " O'Reilly Media, Inc.", **2021**.

- [21] Posit, Shiny - Share your apps, <https://shiny.posit.co/r/getstarted/shiny-basics/lesson7/>, Accessed: 2023-09-01.